

Foreground-only concurrency at streaming scale

How many viewers are **actually watching**, minute by minute — backgrounded and paused time excluded — computed entirely in ClickHouse, gated against raw events, and served in 7 ms.

2,917 ACTUALLY WATCHING AT THE PEAK 2026-07-26 10:56	3,708 WHAT NAIVE SESSION-SPAN COUNTING SAYS A 21.3% OVER-COUNT	17,028 MINUTES RECONCILED AGAINST RAW 0 MISMATCHED	7 ms HEADLINE CURVE, 329 KIB READ FROM SYSTEM.QUERY_LOG
--	--	--	---

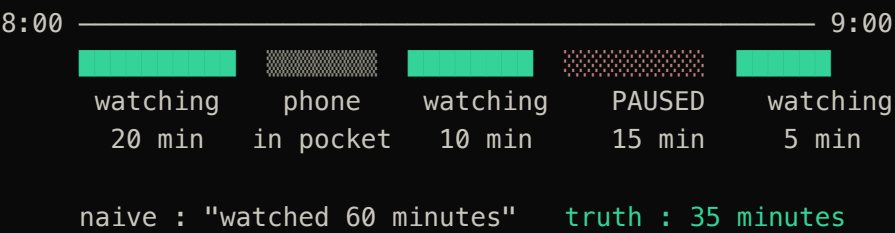
"How many people are watching right now?" — "the most-asked question in the building and one of the hardest to answer correctly."

problem statement

"Ingestion, modeling, and all concurrency computation live in ClickHouse." ·
"Apply foreground-only filtering." · "Publish continuously updated aggregates." · "Hand-computed answers score nothing."

requirements · docs/upstream/, never edited

The app being open is **not** the same as someone watching:



THE FIVE QUESTIONS THE SPEC DEMANDS WE ANSWER WITH WORKING CODE

- 1 What is an **active interval** when the heartbeat is missing, the player paused, the app backgrounded? slide 3
- 2 How are **active ranges represented** – intervals, deltas, hybrid? slide 6
- 3 **Peak and average** without rescanning session history per query? 6 · 7
- 4 **Filter-friendly** across platform / country / content / type / grain? 5 · 11
- 5 **Still-open sessions** whose ranges keep growing? slide 13

MEASURED ON THE PROVIDED FILE

34% of apparent watch time is backgrounded or paused. Foreground-only total: **1,978.1 h**. The whole problem exists to kill that over-count.

"Not watching" happens two ways — and only one is silent

PLAYER STATE	HEARTBEATS / MIN	MEDIAN WINDOW	DETECTABLE BY A GAP THRESHOLD?
actively watching	4.72	—	—
backgrounded	0.047	35 s	YES — a 100× drop; the pings stop
paused	0.756	20 s	NO — one ping every ~79 s slips under any sane threshold



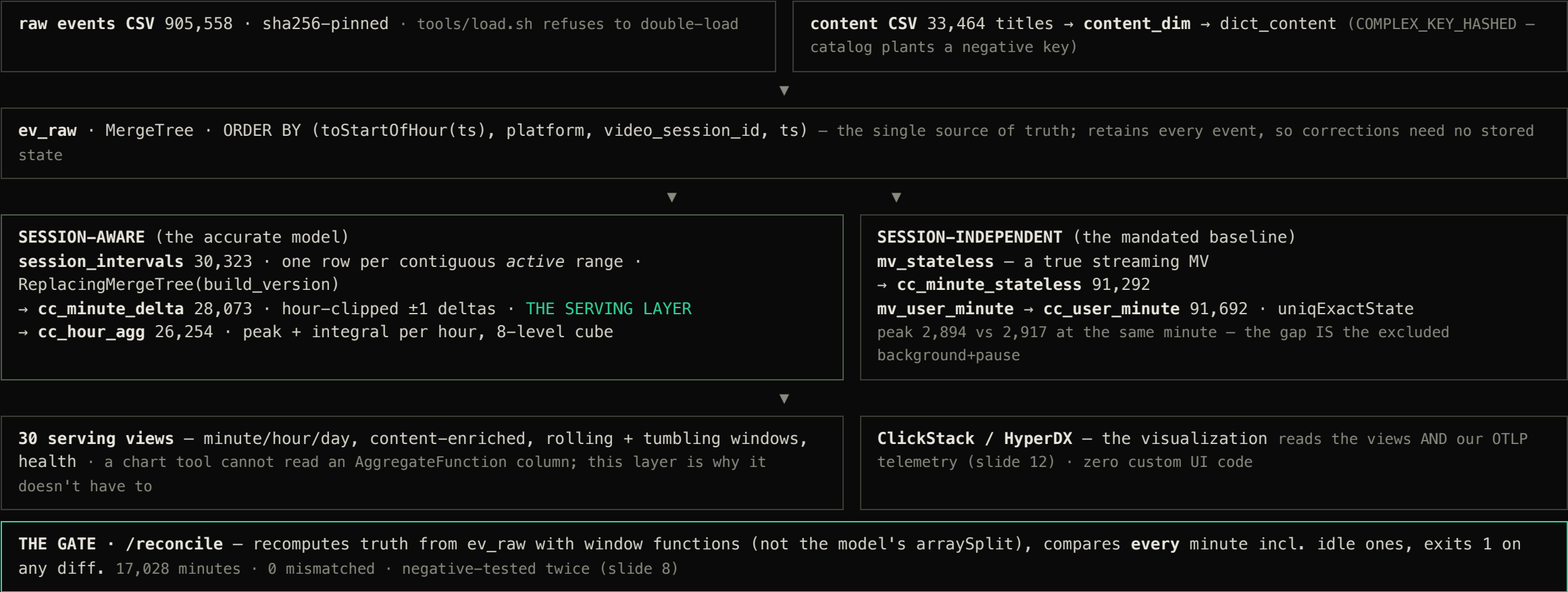
THE HYBRID RULE — MEASURED BEFORE BUILT (ADR 0001, AMENDED BY ADR 0007)

active interval = contiguous run with no gap > 150 s ← catches backgrounding

MINUS explicit [pause, resume) windows ← catches pause — gaps never can

PLUS 60 s tail grace at a run ending in silence

A gap-only model would have shipped, passed every test we own, and silently booked paused time as watching — which the statement explicitly forbids. That is the difference between 3,708 and 2,917 at the peak.



Keys follow the measured access shape — so our two tables sort in opposite directions

Our first key on `ev_raw` led with `video_session_id` "for session locality." The vendored ClickHouse best-practice rules said otherwise; we measured on the real 905K-event file, and the measurement won — over our own decision (ADR 0002).

ACCESS SHAPE	SESSION-FIRST KEY ROWS READ	HOURLY-FIRST KEY ROWS READ	WIN
one-hour time slice	849,888	434,176	2.0×
hour + platform filter (the dashboard shape)	849,888	49,152	17.3×
full interval rebuild	905,558	905,558	identical

The locality argument was worth nothing: the rebuild touches **all** sessions, so it scans everything under either key — while every dashboard query filters by time and usually platform.

AND THE SERVING TABLES INVERT IT

`cc_minute_delta` · ORDER BY (platform, country, content_id, minute, +4 dims) — **dimensions first, time last**, because dashboards filter then range-scan: measured **122×** fewer rows read on the filtered A/B. Two opposite keys, one principle — and every CREATE TABLE carries a comment defending its ORDER BY, per docs/CONVENTIONS.md.

Active ranges become ± 1 deltas, clipped at every hour boundary

A running sum over raw $+1/-1$ deltas needs every delta since the beginning of time — partition pruning becomes decorative. Clipping makes each hour's running sum **absolute and standalone**:

an interval running **20:59:48 → 22:04:49** emits:

```
UNCLIPPED  +1 @20:59 ..... -1 @22:05
            to read 21:30 you must sum from t=0

CLIPPED    hour 20 | +1 @20:59  (survives the hour — no close emitted)
            hour 21 | +1 @21:00  (fresh open)
            hour 22 | +1 @22:00 ... -1 @22:05
            every hour is now absolute — queries prune to the hours they touch
```

PAYOFF 1 — PRUNING IS REAL

Every running sum is `PARTITION BY toStartOfHour(minute)` — a repo-wide convention, because forgetting it yields *plausible wrong numbers*, not errors.

PAYOFF 2 — PEAK PRE-AGGREGATES ACROSS TIME

Per-hour max is a real number, not a fragment — so `cc_hour_agg` stores peak + integral per hour, and a day-grain peak reads **24 rows** per combination instead of 1,440 minutes.

Rejected representation: per-minute explosion of all history (~185M rows vs ~28K — the "only works at hackathon size" choice the statement names). We keep it only as an independent cross-check for the gate, documented as not the serving path.

-State / -Merge, and the three arithmetic rules that keep the numbers honest

RULE	WHAT WE DO	MEASURED COST OF THE NAIVE WAY
Never sum a distinct count	cc_user_minute stores uniqExactState(user_id); reads uniqExactMerge; second hops use -MergeState. Session-level counts are summable across buckets (one session, one platform); user-level never.	9× over-count (45,000 vs truth 5,000)
Peak is not summable across dimensions	Always: filter → sum deltas per minute → running sum within hour → then max(). We never store a per-dimension peak; the 8-level cube computes each curve separately.	+2.0% (platform) +53.6% (content)
uniqExact, never uniq	HyperLogLog's 1–2% error is a silent correctness bug against an exact private answer key.	1–2% silent error

AND THE TYPE THAT IS LOAD-BEARING

SimpleAggregateFunction(sum, Int64) for delta columns. As UInt64, a negated corrective row (slide 13) silently wrapped to 2⁶⁴–n: sum() stayed right by modular arithmetic so the bug hid, while max() returned 1.8e19 and starts inflated 22%. Found by our own truncation test, fixed, re-gated.

Also enforced repo-wide: plain columns in an AggregatingMergeTree must be SimpleAggregateFunction (hard error on 26.7) · dictGet over joining the small dimension (3.7× less memory, measured) · reads may say FINAL, writers never say OPTIMIZE.

minutes_compared=17028 · mismatched=0 · max_abs_diff=0 · PASS

Independent by construction. /reconcile recomputes truth from ev_raw alone using **window functions** — a different algorithm from the model's arraySplit derivation — then compares a **dense minute spine**, so an idle minute can disagree too. A SUMMARY row asserts coverage; silence is failure, not success.

Negative-tested twice — a gate that cannot fail proves nothing:

inject one bad delta row	exit 1
fabricate 500 viewers at an idle minute	25 mismatched · max_abs_diff 500
rebuild	exit 0

And on every rebuild: delta layer vs interval expansion — 3,725 minutes, 0 mismatches · hour tier vs minute tier — 98 hours, 0 · incremental vs from-scratch — 1,578 minutes, 0.

THE STORY THAT MAKES IT CREDIBLE: WE BROKE OUR OWN GATE

An adversarial pass on the first gate found it **passing on zero rows**: five hard-coded 2026-07-26 timestamps meant any other day compared nothing, and grep -q MISMATCH read that silence as success. It was also blind to idle minutes — a fabricated 500 viewers at a quiet minute *passed*.

All three defects fixed and each fix negative-tested. Coverage went **5 → 17,028 minutes**, idle minutes included, on the same zero-mismatch verdict — and the gate now derives its target minutes from the data, so it works unchanged on the unseen day (proven on a holdout: 1,364 minutes, 207 of them idle).

What the gate cannot see — a definitional error, because it recomputes truth from the same spec it is testing. That honest boundary is slide 14.

Every "best practice" was made to prove itself on our data. Six didn't.

CANDIDATE	LOOKED LIKE	MEASURED, ON THE REAL FILE
PROJECTION on ev_raw by session id	27.7× on single-session lookups	the real straggler path is IN (subquery), which full-scans anyway → 1.00× for +94% storage . Not shipped – rejecting a remedy our own ADR had recommended
argMax() instead of FINAL	"FINAL is slow" folklore	3–4× slower, 5.7× more bytes – one active part, zero duplicate keys: FINAL has nothing to collapse
a dedup step (spec step 3)	mandatory hygiene	4,210 duplicate rows are provably inert : raw vs deduped derivation identical – 0 of 3,725 minutes differ (0 of 834 restricted to the 863 duplicate-bearing sessions). "We proved the step unnecessary" beats "we added the step"
any() to attribute dimensions	a cheap tie-break	picks the player's pre-resolution sentinel : 44.5% of audio attributions wrong, 73.5% at the peak minute – and non-deterministic across thread counts. Replaced by dominant-value-per-interval → 3.3%
uniq() (HyperLogLog)	faster distinct	1–2% error against an exact private key – a silent correctness bug
app-side insert batching	standard ingest advice	async_insert=1 already does it server-side; we would reimplement the server and move durability into a process that can die holding data

Not one decision was settled by argument — and **four measurements overturned our own prior plan**, twice reversing a choice we had already ADR'd. The defensible story is not "we designed it well"; it is **"we kept trying to disprove it, and here is the list of times we succeeded."**

We quote what queries read, not just how fast they return

7 ms HEADLINE CONCURRENCY CURVE QUERY CACHE OFF	329 KiB BYTES READ FOR THAT CURVE	28,073 ROWS READ — THE WHOLE DELTA LAYER	8.5× LESS I/O THAN EXPANDING INTERVALS 299 KB VS 2.55 MB
---	---	--	--

Evidence, not vibes. Every benchmark run is tagged `SETTINGS log_comment='...'` and the numbers are read back from `system.query_log` — the authoritative record of latency, rows and bytes. No screenshot, no stopwatch, reproducible by anyone with the service.

A filtered dashboard shape (`ck1_android_phone`): **30 ms · 603 KiB** — still reading the 28K-row delta layer, never raw history.

Why it's cheap by construction: queries read `cc_minute_delta` — rows exist only where concurrency *changes*, hour-clipping bounds every scan to the hours touched, and the dimension-first sort key (slide 5) prunes the rest.

And it's observable: a HyperDX source charts `system.query_log` itself — p95 latency and bytes read of our own queries, on the graded service.

Window views use RANGE frames, not ROWS — a delta table stores only change rows, so "5 rows back" is not "5 minutes back". Rolling/tumbling views verified against a brute-force join: 0 mismatches.

"The provided dataset is a scaled-down proxy for a petabyte-class production problem. Judges will ask how your design behaves at 100×."

problem statement — scale framing

THE HARD ROW CEILING — A BOUND, NOT A HOPE

cc_minute_delta stores at most one open + one close per (merged run, hour), so on this file it is capped at **≤ 36,930 rows no matter how many dimensions exist**:

3 dimensions		24,951 · 67.6%
7 dimensions		28,139 · 76.2%
the ceiling		36,930

Whole table: 111 KiB. Per-minute explosion (~185M rows, ≈6,600× larger) has no such ceiling — that is why it isn't the serving path.

Work scales with **change**, not history: the delta layer is O(intervals), rebuilds touch only sessions that changed, corrections cost O(stragglers). Hour-clipping means no query ever scans from t=0.

WHAT BREAKS FIRST — SAID BEFORE WE'RE ASKED

Publication is batch today. Aggregates rebuild via make model; only two views are true streaming MVs. The fix is validated on this exact service — refreshable MV, 1,579 rows in 8 s, no external scheduler (slide 15).

FINAL vs argMax flips at scale. Our measurement (slide 9) holds at one active part and zero duplicate keys; with many parts and frequent corrections it reverses. We would re-measure, not assume.

The file can't test everything. country has a single value, so a country-filter bug is invisible to every test we own — we test on platform's 10 values instead, and say so.

ClickStack charts our data and observes our pipeline. Delete it and the demo breaks.

DIRECTION 1 — CLICKSTACK IS THE VISUALIZATION

HyperDX sources read our serving views on the graded Cloud service: the concurrency curve, per-dimension drilldowns, content-level views — and a `system.query_log` source charting the p95 latency and bytes read of our own queries. **Zero custom UI code**: the spec puts polished frontends out of scope and separately requires an OSS integration — one component satisfies both.

DIRECTION 2 — CLICKSTACK OBSERVES THE PIPELINE

`sonyiv observe` (Go) emits over OTLP into ClickStack's `otel_*` tables: **watermark lag** — the observable expression of the whole update design — plus build-stage timing and the reconcile gate verdict, rendered as a freshness panel. A ~250-line stdlib-only OTLP/HTTP JSON client; no SDK.

The test we set ourselves — the organiser warns "superficial inclusion won't count": remove ClickStack and the freshness panel has *nowhere to come from*. The integration is load-bearing in both directions, not a logo on a slide.

OSS depth beyond ClickStack: the official ClickHouse/agent-skills best-practice rules (Apache-2.0) are vendored into the repo and cited by name in our ADRs — and they **overturned one of our own schema choices** (the `ev_raw` sort key, slide 5). We treat upstream OSS guidance as a reviewer, not decoration.

Go CLI: `verify (env/schema preflight) · observe (OTLP emit)`. It computes nothing about concurrency — R1 stays intact: all modeling and computation live in ClickHouse.

Nothing is ever mutated. Late data is absorbed, appended, or diffed — never rebuilt.

ARRIVAL CLASS	MECHANISM	WHY IT IS CORRECT
newer than watermark $W = 2,400$ s (set by the measured 2,081 s straggler tail, not guessed)	hot tier: heartbeat leases + <code>uniqExact</code> (ADR 0004/0005)	idempotent and monotone — replays are no-ops, late beats are pure additions; open sessions just keep renewing leases
in normal order	finalizer: re-derive only sessions touched since last run	appends sealed hour-clipped deltas behind W ; cost scales with changed sessions, not history
older than W (straggler)	correction-by-diff (ADR 0006): recompute that one session with and without, append the difference	exactly as correct as a full rebuild — because it <i>is</i> one, of one session. <code>SimpleAggregateFunction(sum, Int64)</code> absorbs the negative rows natively

THE PROOF — A TRUNCATION TEST THAT MANUFACTURES THE WORST CASE

Cut the stream at the peak minute — **52.6% of sessions left open** — then absorb the withheld **447,081 late events** incrementally: never truncating, never mutating, only appending. Result: row-for-row equal to a from-scratch build on all **1,578 minutes, 0 mismatches**.

It caught **two real bugs** before any judge could: a stale-row defect (versioning intervals by `interval_end` let an overshoot tail-grace win forever — 316 intervals, peak over-counted by 37; now `ReplacingMergeTree(build_version)`) and the `UInt64` wrap on negated corrections (slide 7). Both fixed, both re-gated.

Honest status: the hot tier and the live straggler demo are designed and ADR'd, not yet wired. The truncation test is the proof of absorption; the live path is next (slide 15).

Four definitions the data cannot settle — each measured, each built as a switch

FORK	OUR DEFAULT, STATED	MEASURED WORTH
1 · resume semantics. 9,958 resume→resume pairs; 900 sessions whose first pause/resume event is a resume — a clean toggle cannot produce that. Does a pause close at the first resume, or one that corresponds to it?	close at first resume; unpaired resumes are noise	189.2 h · 9.7% of counted watch time
2 · unclosed pauses. 23% of pauses never resume. Stay paused to end of run, or end at next event?	conservative — stays paused (under-counting is visible and explainable; over-counting invents viewers)	99.3 h · 5.09% +131 (+4.5%) on the graded peak
3 · heartbeat cadence. The doc says 60 s; the shipped data ticks at 40.0 s (p50 = p90 on three separate telemetry streams). Which did the ground truth assume?	the shipped data is authoritative	≤ 170.9 h · 8.8% upper bound
4 · is Hindi one language or four? hin / HIN / hin-hindi / hin-Hindi are four raw values.	store raw, normalise on read — never degrade the derivation	23.3% on any per-language answer (peak 1,768 → 2,180)

None of these is measurable from the data: the ground truth is private, so a wrong guess is silently wrong on every benchmark answer. Our gate **structurally cannot catch them** — it recomputes truth from the same rule, so it agrees with itself by construction. That is why each fork is a **one-line switch** (e.g. UNCLOSED_PAUSE_TO_RUN_END) with an ~11 s rebuild: the moment a definition is pinned, we re-run and re-gate.

Also disclosed: country carries a single value in the provided file, so country-filter bugs are invisible to our tests — we validate filtering on platform's 10 values and say so out loud.

Next steps, in scoring order — each already de-risked

STEP	STATE OF DE-RISKING
1 · Continuous publication — the spec's "continuously updated aggregates", our biggest stated gap	native refreshable materialized views, already validated on the graded service: 1,579 rows self-populated in 8 s , no external scheduler. The refresh interval becomes a declared freshness SLA, with lag observable in <code>system.view_refreshes</code> and charted in ClickStack
2 · Wire the hot tier , stitched to the sealed tier at W	designed and ADR'd (0004/0005); upgrades update handling from "proven by test" to "watch it live", and unblocks the stitch-boundary probe
3 · Live straggler demo	the arithmetic is already proven by the truncation test (1,578 minutes, 0 mismatches); only the live path remains
4 · Tail-sensitivity sweep — gap {120, 150, 180} × tail {0, 60, 150}	robustness evidence: either the answer is stable, or we show exactly the point we knowingly chose
5 · /bench on the released query set	harness ready — <code>log_comment</code> -tagged latency + bytes read from <code>system.query_log</code>
6 · Ingest definition answers (slide 14)	every fork is a one-line switch and an ~11 s rebuild, re-gated by <code>/reconcile</code>

THE CLOSE

Correct first — 17,028 minutes gated against raw events, negative-tested. **Fast second** — 7 ms and 329 KiB for the headline curve, and we quote what queries *read*. **Update-friendly third** — three arrival classes, three mechanisms, absorption proven by a test that caught two real bugs. And where a definition is genuinely open, we measured what it is worth and built the switch.